Spark-MLeap Fraud Detection Project

Technical Interview Preparation Guide & Expected Q&A;

Section 1: Architecture & Design Decisions

Q1: Walk me through the high-level architecture of this fraud detection system.

A: The system follows a decoupled, low-latency machine learning architecture. It consists of three primary components: (1) **Offline Model Training Pipeline** in PySpark, which performs feature engineering (string indexing, vector assembly) and trains a Random Forest model. (2) **MLeap Serving Microservice** (a containerized Scala/Spring Boot app) that loads the serialized model and performs real-time, low-latency predictions. (3) **FastAPI API Gateway**, which accepts client HTTP requests, formats them into LeapFrames, posts them to the MLeap server, logs results to a local SQLite database using SQLAlchemy, and tracks latencies.

Q2: Why did you decouple model training (PySpark) from serving (MLeap)?

A: Decoupling solves the 'latency vs scale' challenge. PySpark is great for large-scale distributed training, but starting a SparkSession or executing Spark pipelines for real-time single-row inference introduces significant overhead (JVM startup, execution planning, distributed overhead), often resulting in 100ms+ latency. MLeap allows us to serialize the entire Spark ML pipeline (including transformations) into a single bundle and run it on a lightweight, native Scala runtime. This drops inference latency down to under 15ms, which is critical for real-time transactional fraud scoring.

Q3: What are the benefits of having an API Gateway (FastAPI) in front of the MLeap service?

A: The API Gateway acts as a reverse proxy and abstraction layer. It provides: (1) **Separation of concerns**: MLeap focuses purely on running the ML bundle; FastAPI handles routing, logging, database persistence, and API formatting. (2) **Security & Authentication**: Gateway endpoints can be easily protected with API keys or OAuth2 tokens. (3) **Data validation**: Utilizing Pydantic schemas ensures that input parameters are validated before reaching downstream servers.

Section 2: PySpark & Feature Engineering

Q4: Explain your feature engineering pipeline. Why did you use StringIndexer and VectorAssembler?

A: Our model uses both categorical features (merchant category, device type) and numerical features (transaction amount, hour of day). We use **StringIndexer** to encode the categorical strings into numeric indices (e.g. 'Electronics' becomes 0.0, 1.0, etc.), handling unseen classes using 'keep' as the invalid category strategy. Then, **VectorAssembler** compiles these numerical columns into a single vector column ('features') required by Spark's ML algorithms. These stages are integrated into a single Spark **Pipeline** so that they are exported and applied identically in production during inference.

Q5: How does PySpark handle model training internally, and how is it optimized?

A: PySpark coordinates distributed data processing via a Driver and worker Executors. In our local setup, we use local Spark mode. The Random Forest training is parallelized across trees and nodes. We used PySpark 3.3.2 and configured custom package dependency ('ml.combust.mleap:mleap-spark_2.12:0.20.0') to provide MLeap serialization support directly in Spark's classpath, ensuring seamless serialization of custom pipeline stages.

Section 3: MLeap Serialization & Serving

Q6: How does model serialization with MLeap work? What is exported in 'model.zip'?

A: MLeap uses a custom serialization format. It writes the metadata, schema, and parameters of each pipeline stage (the StringIndexers, VectorAssembler, and Random Forest model) into JSON/Protobuf files inside a directory structure, which is then zipped into `model.zip` (the MLeap Bundle). This bundle contains all instructions required to recreate the exact transformation and classification steps in the target MLeap execution environment, completely independent of a live Spark cluster.

Q7: How does the MLeap server perform inference? What is a LeapFrame?

A: MLeap defines a data format called a **LeapFrame** (analogous to a Spark DataFrame or Pandas DataFrame). A LeapFrame contains a schema defining fields (names and types) and the actual data rows. The FastAPI gateway constructs this JSON LeapFrame from the client request, sends it to MLeap's `/transform` REST endpoint, and MLeap uses its native engine to execute the serialized stages in-memory and return a new LeapFrame containing prediction results and class probability vectors.

Section 4: API Design & Metadata Persistence

Q8: How did you implement logging, and why is SQLite used here?

A: In the FastAPI gateway (`app.py`), we configured SQLAlchemy ORM to communicate with an SQLite database (`transactions.db`). On each `/predict` call, once the prediction and probability score are retrieved from the MLeap server, we calculate the API processing latency and save a new `TransactionLog` record. SQLite is ideal for a local proof-of-concept due to its zero-configuration setup, file-based persistence, and standard SQL support. In production, this would be swapped for a distributed database like PostgreSQL or Cassandra.

Q9: What security risks are present in this architecture, and how would you address them?

A: (1) **Lack of Authentication**: The endpoints are currently open. In production, we would integrate OAuth2/JWT tokens in FastAPI. (2) **Model Bundle Integrity**: The model is loaded via a local path (`file:/models/model.zip`). If attackers overwrite this bundle, they can execute malicious logic or inject biased models. We should protect the bundle storage (e.g. secure S3 buckets) and use signature validation (SHA-256) before loading. (3) **SQL Injection**: We use SQLAlchemy ORM, which parameterized queries by default, protecting us from basic SQL injection attacks.

Section 5: System Design & Scaling to Production

Q10: If this API handles 50,000 requests per second, how would you scale this system?

A: To scale to that volume:
• **Load Balancing & Replica Scaling**: Deploy the FastAPI Gateway and MLeap services in a Kubernetes cluster, scaling the pods horizontally based on CPU/traffic, behind a load balancer (e.g., NGINX, AWS ALB).
• **Asynchronous Database Logging**: Writing synchronously to SQLite will block API threads at high RPS. We should replace SQLite with a message queue like **Apache Kafka** or RabbitMQ. FastAPI can publish prediction logs asynchronously to Kafka, and a downstream consumer group can digest and write the logs to a data lake or highly scalable database (like Cassandra or DynamoDB).
• **Protocol Optimization**: Switch from REST/HTTP JSON communication between FastAPI and MLeap to **gRPC/Protobuf**, which significantly reduces CPU usage and payload sizes.
• **Caching**: Add a Redis cache layer for frequent transactions or pre-computed user risks to bypass the model serving layer entirely when applicable.